PySpark Functions

A Complete Guide for Data Engineers

Filter • Join • Window • Aggregate • String • Date • UDF • SQL & more

DataFrame API | Spark SQL | Streaming | MLlib | Data Engineering

What We'll Cover

10 categories of PySpark functions every data engineer must know

01 DataFrame Basics
`SparkSession, read, show, schema`

02 Filter & Select
`filter(), where(), select(), col()`

03 GroupBy & Aggregations
`groupBy(), agg(), count(), sum()`

04 Join Functions
`inner, left, right, outer, cross, semi`

05 Window Functions
`rank(), lag(), lead(), row_number()`

06 String Functions
`concat(), split(), regexp(), trim()`

07 Date & Time
`to_date(), datediff(), date_trunc()`

08 Math & Stats
`round(), abs(), stddev(), corr()`

09 UDFs
`udf(), pandas_udf(), vectorized UDFs`

10 I/O & SQL
`read/write formats, spark.sql()`

01 — DataFrame Basics

SparkSession initialization, reading data, and inspecting DataFrames

▸ *SparkSession & Reading Data*

```
from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType,
IntegerType

# Initialize SparkSession
spark = SparkSession.builder \
    .appName("DataEngineering") \
    .config("spark.sql.adaptive.enabled", "true") \
    .getOrCreate()

# Read data in various formats
df_parquet = spark.read.parquet("s3://bucket/data/")
df_csv     = spark.read.option("header", True).csv("data.csv")
df_json    = spark.read.json("data.json")
df_jdbc    = spark.read.jdbc(url, "table", properties=props)
```

df.cache()

Persist DataFrame in memory for reuse in iterative jobs

df.repartition(n)

Change number of partitions. Use before wide transformations

df.coalesce(n)

Reduce partitions (no shuffle). Ideal before writing output

df.explain()

Print logical & physical query plan for optimization insight

▸ *Inspect & Create*

```
# Schema inspection
df.printSchema()          # Tree-format
schema
df.dtypes                 # List of (name,
dtype)
df.schema                 # StructType
object

# Data preview
df.show(5, truncate=False) # Show 5 rows
df.display()               # Databricks
only
df.count()                 # Row count
df.columns                 # Column names
list

# Create from data
from pyspark.sql import Row
df = spark.createDataFrame([
    Row(name="Alice", age=30),
    Row(name="Bob",   age=25)
])
```

02 — Filter, Select & Column Operations

Subsetting rows and transforming columns

▸ *Filter / Where*

```
from pyspark.sql.functions import col, lit, expr, when, coalesce

# ── FILTERING ROWS ──────────────────────
df.filter(col("age") > 25)
df.filter("age > 25")                    # SQL string
df.where(col("status") == "active")    # alias of filter
df.filter((col("age") > 25) & (col("dept") == "Eng"))

# Multiple conditions
df.filter(col("city").isin(["NYC", "LA", "Chicago"]))
df.filter(col("name").isNotNull())
df.filter(col("score").between(70, 100))
df.filter(col("email").rlike(r"@gmail\.com$"))
```

▸ *Select & withColumn*

```
# ── SELECTING COLUMNS ────────────────
df.select("name", "age")
df.select(col("name"), col("age") + 1)
df.selectExpr("name", "age * 2 as
double_age")

# ── COLUMN TRANSFORMATIONS ──────────────
df.withColumn("senior", col("age") >= 60)
df.withColumn("full_name", col("first") + "
" + col("last"))
df.withColumnRenamed("old_name", "new_name")
df.drop("unused_col")

# WHEN / CASE WHEN
df.withColumn("tier",
  when(col("score") >= 90, "Gold")
  .when(col("score") >= 70, "Silver")
  .otherwise("Bronze"))
```

col() vs expr()

col() references a column. expr() evaluates SQL string expressions with full SQL syntax.

coalesce()

Returns first non-null value: coalesce(col('a'), col('b'), lit(0))

alias()

Rename a column expression: col('age').alias('user_age')

cast()

Convert types: col('price').cast('double') or cast(DoubleType())

03 — GroupBy & Aggregations

Summarizing data with groupBy, pivot, and aggregate functions

▸ *groupBy & agg*

```
from pyspark.sql.functions import (
    count, countDistinct, sum, avg, mean,
    min, max, collect_list, collect_set,
    first, last, stddev, variance, percentile_approx
)

# ── BASIC GROUPBY ──────────────────────
df.groupBy("department") \
  .agg(count("*").alias("total"),
       avg("salary").alias("avg_salary"),
       max("salary").alias("max_salary"))

# ── MULTIPLE GROUP KEYS ────────────────
df.groupBy("dept", "year") \
  .agg(sum("revenue").alias("total_rev"),
       countDistinct("customer_id").alias("uniq_cust"))
```

count(*)

Count all rows incl nulls

sum_distinct()

Sum of distinct values

countDistinct()

Count unique non-null values

approx_count_distinct()

Fast approximate distinct count (HyperLogLog)

▸ *Pivot, Collect, Stats*

```
# ── PIVOT TABLE ────────────────────────
df.groupBy("dept") \
  .pivot("quarter", ["Q1","Q2","Q3","Q4"]) \
  .agg(sum("sales"))

# ── COLLECT INTO LIST/SET ──────────────
df.groupBy("user_id") \
  .agg(collect_list("product").alias("produc
ts"),

collect_set("category").alias("categories"))

# ── PERCENTILE & STATS ─────────────────
df.groupBy("region") \
  .agg(
    percentile_approx("value",
0.5).alias("median"),
    stddev("value").alias("std_dev"),
    variance("value").alias("variance")
  )
```

04 — Join Functions

Combining DataFrames with all join types

▸ *Join Syntax*

```
# ── ALL JOIN TYPES ──────────────────
# inner  : matching rows in both
# left   : all left + matching right
# right  : all right + matching left
# outer/full : all rows from both
# left_semi  : left rows with a match in right
# left_anti  : left rows WITHOUT match in right
# cross      : cartesian product

orders.join(customers, on="customer_id", how="inner")
orders.join(customers, on=["cust_id","region"], how="left")

# Join with different column names
orders.join(customers,
    orders.cust_id == customers.customer_id,
    how="left_outer")
```

▸ *Broadcast & Advanced*

```
# ── BROADCAST JOIN (small table) ───
from pyspark.sql.functions import broadcast

large_df.join(broadcast(small_df),
              on="key", how="inner")

# ── AVOIDING DUPLICATE COLUMNS ──────
# Drop before join or use alias
orders.alias("o").join(
    customers.alias("c"),
    col("o.cust_id") == col("c.id")
).select("o.*", "c.name", "c.email")

# ── CROSS JOIN ──────────────────────
df1.crossJoin(df2)
df1.join(df2, how="cross")
```

Join Type	Returns	Use Case
inner	Matching rows only	Most common — clean merge
left	All left + matched right	Keep all source records
right	Matched left + all right	Keep all target records
full outer	All rows from both sides	Reconciliation / diff
left_semi	Left rows with match (no cols)	Existence filter
left_anti	Left rows WITHOUT match	Find orphan records

05 — Window Functions

Perform calculations across related rows without collapsing the DataFrame

▸ *Window Spec & Ranking*

```
from pyspark.sql.functions import (
    row_number, rank, dense_rank,
    lag, lead, first, last,
    sum, avg, min, max,
    ntile, percent_rank, cume_dist
)
from pyspark.sql.window import Window

# ── DEFINE WINDOW SPEC ────────────────
w = Window.partitionBy("dept") \
          .orderBy(col("salary").desc())

# ── RANKING FUNCTIONS ────────────────
df.withColumn("row_num",    row_number().over(w))
df.withColumn("rank",       rank().over(w))
df.withColumn("dense_rank", dense_rank().over(w))
df.withColumn("ntile_4",    ntile(4).over(w))
```

rowsBetween()
Physical row offsets: Window.unboundedPreceding, 0, Window.unboundedFollowing

rangeBetween()
Logical value range on ORDER BY column. Great for time-series rolling windows.

partitionBy()
Divides data into groups. Each partition is processed independently.

cume_dist()
Cumulative distribution: fraction of rows ≤ current row in partition.

▸ *Lag/Lead & Running Calcs*

```
# ── LAG / LEAD (offset access) ──────
w_ordered = Window.partitionBy("user") \
                  .orderBy("event_date")

df.withColumn("prev_val", lag("value",
1).over(w_ordered))
df.withColumn("next_val", lead("value",
1).over(w_ordered))

# ── RUNNING TOTALS / MOVING AVG ─────
w_running = Window.partitionBy("dept") \
    .orderBy("date") \
    .rowsBetween(Window.unboundedPreceding,
0)

df.withColumn("running_sum",
sum("sales").over(w_running))

# ── RANGE WINDOW ────────────────────
w_range = w_ordered.rangeBetween(-6, 0)
df.withColumn("7d_avg",
avg("value").over(w_range))
```

06 — String Functions

Text manipulation, pattern matching, and string transformations

▸ *Concat, Split & Case*

```
from pyspark.sql.functions import (
    concat, concat_ws, split, substring,
    length, trim, ltrim, rtrim,
    upper, lower, initcap,
    regexp_replace, regexp_extract,
    like, rlike, contains,
    lpad, rpad, repeat, reverse,
    translate, overlay, format_string
)

# ── CONCAT & SPLIT ──────────────────
df.withColumn("full_name", concat(col("first"), lit(" "),
col("last")))
df.withColumn("full_name", concat_ws(" ", "first", "middle",
"last"))
df.withColumn("parts", split(col("csv_col"), ","))
df.withColumn("domain", split(col("email"), "@")[1])
```

▸ *Regex & Formatting*

```
# ── PATTERN MATCHING ────────────────
df.filter(col("name").rlike("^[A-Z].*"))
df.withColumn("area_code",
    regexp_extract(col("phone"), r"^(\
d{3})", 1))
df.withColumn("clean",
    regexp_replace(col("text"), r"[^a-zA-Z0-
9]", ""))

# ── SUBSTRING & PADDING ─────────────
df.withColumn("year",
substring(col("date_str"), 1, 4))
df.withColumn("id_padded", lpad(col("id"),
8, "0"))

# ── FORMATTING ──────────────────────
df.withColumn("label",
    format_string("User: %s | Score: %.2f",
                  col("name"),
col("score")))
```

trim/ltrim/rtrim

Remove whitespace from both, left, or right side of string

upper/lower/initcap

Case conversion: ALL CAPS, all lower, or Title Case

translate()

Character-level replacement: translate(col, 'abc', 'ABC')

levenshtein()

Edit distance between two strings. Useful for fuzzy matching

07 — Date & Time Functions

Parsing, formatting, and arithmetic on dates and timestamps

▸ *Parsing & Converting*

```
from pyspark.sql.functions import (
    current_date, current_timestamp,
    to_date, to_timestamp, from_unixtime, unix_timestamp,
    date_format, date_trunc,
    year, month, dayofmonth, dayofweek, dayofyear,
    hour, minute, second, quarter,
    datediff, months_between, add_months,
    date_add, date_sub, next_day, last_day
)

# ── PARSING & CONVERTING ──────────────
df.withColumn("dt", to_date(col("str_date"), "yyyy-MM-dd"))
df.withColumn("ts", to_timestamp(col("str_ts"), "yyyy-MM-dd
HH:mm:ss"))
df.withColumn("epoch", unix_timestamp(col("ts")))
df.withColumn("from_epoch", from_unixtime(col("epoch")))
```

date_trunc()

Truncate to 'year', 'month', 'week', 'day', 'hour', 'minute'

window()

Time-based grouping for streaming: window(ts, '1 hour', '30 minutes')

last_day()

Returns the last day of the month for a given date column

next_day()

Next specified weekday after date: next_day(col, 'Mon')

▸ *Extract & Arithmetic*

```
# ── EXTRACTION ────────────────────────
df.withColumn("yr",  year("event_date"))
df.withColumn("mo",  month("event_date"))
df.withColumn("day",
dayofmonth("event_date"))
df.withColumn("dow",
dayofweek("event_date")) # 1=Sun
df.withColumn("qtr", quarter("event_date"))

# ── DATE ARITHMETIC ───────────────────
df.withColumn("diff",   datediff(col("end"),
col("start")))
df.withColumn("months",
months_between(col("end"), col("start")))
df.withColumn("future", date_add(col("dt"),
30))
df.withColumn("trunc",  date_trunc("month",
col("ts")))

# ── FORMATTING ────────────────────────
df.withColumn("fmt", date_format(col("ts"),
"dd/MM/yyyy"))
```

08 — Math & Statistical Functions

Numeric operations, rounding, and statistical analysis

▸ *Rounding & Numeric*

```
from pyspark.sql.functions import (
    abs, round, ceil, floor, bround,
    sqrt, cbrt, exp, log, log2, log10, log1p,
    pow, factorial, signum, pmod,
    sin, cos, tan, asin, acos, atan, atan2,
    degrees, radians, hypot,
    greatest, least, nanvl
)

# ── BASIC MATH ──────────────────────
df.withColumn("rounded",   round(col("price"), 2))
df.withColumn("ceiling",   ceil(col("score")))
df.withColumn("floored",   floor(col("score")))
df.withColumn("absolute",  abs(col("delta")))
df.withColumn("power",     pow(col("base"), 2))
df.withColumn("root",      sqrt(col("area")))
```

bround()
Banker's rounding (round half to even) — avoids systematic bias

nanvl()
Returns first non-NaN value: nanvl(col('a'), lit(0))

log1p()
log(1 + x) — numerically stable for small values. Common in ML features

pmod()
Positive modulo: always returns non-negative result unlike % operator

▸ *Statistical Functions*

```
from pyspark.sql.functions import (
    corr, covar_pop, covar_samp,
    kurtosis, skewness,
    percentile_approx, approx_count_distinct
)

# ── STATISTICAL AGGREGATIONS ────────
df.agg(
    corr("x", "y").alias("correlation"),
    covar_samp("x",
"y").alias("covariance"),
    kurtosis("value").alias("kurtosis"),
    skewness("value").alias("skewness"),
    percentile_approx("amount",
        [0.25, 0.5,
0.75]).alias("quartiles")
)

# ── COLUMN-WISE COMPARISONS ─────────
df.withColumn("max_val", greatest(col("a"),
col("b"), col("c")))
df.withColumn("min_val", least(col("a"),
col("b"), col("c")))
```

09 — User Defined Functions (UDFs)

Extend PySpark with custom Python logic — and how to use them efficiently

▸ *Standard UDF*

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType, IntegerType

# ── STANDARD UDF ──────────────────────
def categorize(score):
    if score >= 90: return "A"
    elif score >= 80: return "B"
    else: return "C"

grade_udf = udf(categorize, StringType())
df.withColumn("grade", grade_udf(col("score")))

# ── DECORATOR SYNTAX ─────────────────
@udf(returnType=StringType())
def clean_name(name):
    return name.strip().title() if name else None

df.withColumn("clean", clean_name(col("name")))
```

UDF Type	Serialization	Speed	Best Use Case
Standard UDF	Row-by-row Python pickled	Slowest	Simple row transforms
Pandas UDF	Apache Arrow (columnar)	Fast	Vectorized math/ML
Grouped Map	Apache Arrow per group	Fast	Group-level transforms
Native Spark fn	JVM — no Python overhead	Fastest	Always prefer if available

▸ *Pandas UDF (Vectorized)*

```
from pyspark.sql.functions import pandas_udf
from pyspark.sql.types import DoubleType
import pandas as pd

# ── PANDAS UDF (Vectorized) ───────────
# Much faster than standard UDFs!
@pandas_udf(DoubleType())
def zscore_udf(series: pd.Series) ->
pd.Series:
    return (series - series.mean()) /
series.std()

df.withColumn("zscore",
zscore_udf(col("value")))

# ── GROUPED MAP PANDAS UDF ────────────
from pyspark.sql.functions import
PandasUDFType

@pandas_udf(df.schema,
PandasUDFType.GROUPED_MAP)
def normalize_group(pdf: pd.DataFrame) ->
pd.DataFrame:
    pdf["norm"] = (pdf["value"] -
pdf["value"].min()) \
                / (pdf["value"].max() -
pdf["value"].min())
    return pdf

df.groupBy("dept").apply(normalize_group)
```

10 — Array, Map & Struct Functions

Working with complex nested data types in PySpark

▸ *Array Functions*

```
from pyspark.sql.functions import (
    array, array_contains, array_distinct,
    array_except, array_intersect, array_union,
    array_join, array_max, array_min,
    array_remove, array_sort, array_zip,
    explode, explode_outer, posexplode,
    flatten, size, slice, sequence,
    transform, filter, aggregate, zip_with
)

# ── ARRAY OPERATIONS ──────────────────
df.withColumn("arr", array(col("a"), col("b"), col("c")))
df.withColumn("has_py", array_contains(col("skills"), "Python"))
df.withColumn("sorted", array_sort(col("tags")))
df.withColumn("unique", array_distinct(col("items")))
df.withColumn("joined", array_join(col("tags"), ", "))
```

transform()

Apply lambda to each array element: transform(col('arr'), lambda x: x*2)

filter(arr, fn)

Keep array elements matching condition: filter(col('nums'), lambda x: x > 0)

aggregate()

Reduce array to scalar: aggregate(col('arr'), lit(0), lambda acc,x: acc+x)

struct()

Create struct type: struct(col('a'), col('b')). Access with col('s.a')

▸ *Explode & Map Functions*

```
# ── EXPLODE (flatten nested) ──────────
# explode: drops rows with null/empty
df.withColumn("tag", explode(col("tags")))

# explode_outer: keeps rows with null
df.withColumn("tag",
explode_outer(col("tags")))

# posexplode: adds index column
df.select("id",
posexplode(col("items")).alias("pos","item")
)

# ── MAP FUNCTIONS ────────────────────
from pyspark.sql.functions import (
    create_map, map_keys, map_values,
    map_contains_key, element_at,
    map_from_arrays, map_concat, map_filter
)
df.withColumn("m", create_map(lit("k"),
col("v")))
df.withColumn("keys",
map_keys(col("props")))
df.withColumn("val",
element_at(col("props"), "color"))
```

11 — I/O Functions & Spark SQL

Reading, writing data and running SQL queries on DataFrames

▸ *Read & Write*

```
# ── READ FORMATS ──────────────────────
spark.read.parquet("path/")
spark.read.orc("path/")
spark.read.json("path/")
spark.read.csv("path/", header=True, inferSchema=True)
spark.read.format("delta").load("path/")
spark.read.jdbc(url, "schema.table", properties=props)
spark.read.table("catalog.schema.table")   # Unity Catalog

# ── WRITE FORMATS ─────────────────────
df.write.mode("overwrite").parquet("output/")
df.write.mode("append").partitionBy("year","month") \
  .parquet("output/")
df.write.format("delta").saveAsTable("my_table")
df.write.jdbc(url, "table", mode="append", properties=props)
df.coalesce(1).write.csv("output/", header=True)
```

Write Modes

overwrite, append, ignore, error (default). Use errorIfExists for safety

partitionBy()

Partition output files by column values. Enables partition pruning on read

bucketBy()

Pre-shuffle data into fixed buckets. Speeds up repeated joins on bucketed columns

sortBy()

Sort within buckets. Use with bucketBy() for max join optimization

▸ *Spark SQL*

```
# ── SPARK SQL ────────────────────────
# Register temp view
df.createOrReplaceTempView("orders")
df.createOrReplaceGlobalTempView("orders") #
cross-session

# Query with SQL
result = spark.sql("""
    SELECT dept,
           COUNT(*) as cnt,
           AVG(salary) as avg_sal
    FROM orders
    WHERE year = 2024
    GROUP BY dept
    ORDER BY avg_sal DESC
""")

# ── SQL CATALOG ──────────────────────
spark.catalog.listTables()
spark.catalog.listDatabases()
spark.catalog.cacheTable("orders")
spark.catalog.refreshTable("orders")
```

12 — Structured Streaming Functions

Real-time data processing with PySpark Structured Streaming

▸ *Kafka Source & Windowing*

```
# ── READ STREAM ──────────────────────
stream_df = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "broker:9092") \
    .option("subscribe", "events") \
    .option("startingOffsets", "latest") \
    .load()

# Decode Kafka message
from pyspark.sql.functions import from_json, col
schema = StructType([...])
parsed = stream_df.select(
    from_json(col("value").cast("string"), schema).alias("data")
).select("data.*")

# ── WATERMARK & WINDOW ───────────────
windowed = parsed \
    .withWatermark("event_time", "10 minutes") \
    .groupBy(window(col("event_time"), "5 minutes"), "user_id") \
    .agg(count("*").alias("event_count"))
```

append mode

Only new rows added since last trigger are written. Cannot update past results.

update mode

Only updated rows written. More efficient than complete for most aggregations.

complete mode

Entire result table written every trigger. Only with aggregation queries.

▸ *Write Stream & Triggers*

```
# ── WRITE STREAM ─────────────────────
# Output modes: append, complete, update
query = windowed.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation",
"/chk/path") \
    .trigger(processingTime="30 seconds") \
    .start("output/path")

# ── OTHER TRIGGERS ───────────────────
.trigger(once=True)          # run once
.trigger(availableNow=True)  # process all
available
.trigger(continuous="1 second") # low-
latency

# ── FOREACH BATCH ────────────────────
def process_batch(batch_df, batch_id):
    batch_df.write.format("jdbc").save()

query = df.writeStream \
    .foreachBatch(process_batch) \
    .start()
```

Quick Reference Cheat Sheet

Most commonly used PySpark functions at a glance

Category	Key Functions
🔍 Filter	filter(), where(), isin(), isNull(), isNotNull(), between(), rlike()
📋 Select	select(), selectExpr(), withColumn(), withColumnRenamed(), drop(), alias()
📊 Aggregation	groupBy(), agg(), count(), sum(), avg(), min(), max(), collect_list()
🔗 Join	join(how=), broadcast(), crossJoin(), left_semi, left_anti
🪟 Window	Window.partitionBy().orderBy(), row_number(), rank(), lag(), lead(), sum().over()
📝 String	concat_ws(), split(), regexp_replace(), regexp_extract(), upper(), lpad()
📅 Date/Time	to_date(), to_timestamp(), datediff(), date_trunc(), year(), month()
🔢 Math	round(), ceil(), floor(), sqrt(), pow(), abs(), greatest(), percentile_approx()
📦 Array/Map	explode(), array_contains(), transform(), filter(), map_keys(), element_at()
💾 I/O	read.parquet/csv/json/delta, write.mode().partitionBy(), spark.sql()
⚡ UDF	udf(fn, ReturnType), @pandas_udf(dtype), PandasUDFType.GROUPED_MAP
🌊 Streaming	readStream, writeStream, watermark(), window(), foreachBatch(), trigger()

You're ready to engineer at scale with PySpark!

From filtering rows to streaming pipelines — you now have a complete function reference for building production-grade data engineering solutions with Apache Spark.

📚 Resources: spark.apache.org/docs • PySpark API Reference • Databricks Documentation • GitHub: apache/spark